

Student Name:

Weight: 15%

Student ID:

Marks: /100

Assignment: Programming Basics

Type: Group Assignment

Needed Modules: 1 and 2 ONLY

- Students should **ONLY USE** programming constructs covered in modules 1 and 2.
- **Late submission will not be accepted**

Scenario

The IT department at one of the universities wanted to keep a record of all the students at the department with their accumulative GPAs. The head of the department hired you to create a program which add students to the program with their GPAs and can print all the students and their GPAs.

Equipment and Materials

For this assignment, you will need:

- Python IDE
- Git/GitHub

Instructions

This assignment consists of the following sections, all completed outside of class time. See the Brightspace for exact due date.

1. Working in a group, review the Scenario and the Grade Registry Program details sections of this document. (Planning, Analysis)
2. Draft a flowchart to represent the process/algorithm before writing the code of the program. (Design)
3. Write a program that meets the requirements described in the scenario above and prints the list of students with their corresponding accumulative GPAs. (Implementation)
4. When your program is complete, use the data listed in each of the following three tests below to see if your program works correctly. (Testing)

Note: Check your program against the marking criteria for the submission.

5. Your program's output **MUST EXACTLY MATCH** the following tests.

6. Submit the following to Brightspace:

- The code of the program that you implemented (.py file)
- A copy of the output from your three test runs (.txt file)
- The flow chart in PDF format.
- Your GitHub repository for this assignment.

Grade Registry Program

- The program uses the dictionary data structure to store students with their corresponding accumulative grades.
- The program asks the user if they want to add a student, if user answer with “yes” then the program asks the user to enter the GPAs for the student’s courses. If the user answers with “no”, the program prints the students’ names and exists.
- If the user enters -1, it means the user has finished inputting all the GPAs for the current student.
- After the user inputs all the GPAs, the average accumulative GPA is computed from the all the GPAs for all the courses taken by the student.
- If the user does not enter any GPA, it means a student is in the first semester and does not have any GPAs yet.
- Please use python lists with the append function to add new GPAs to the GPAs list. Obviously, there will be one list for each student. You can reuse the same list again for new students.
- Put the whole algorithm in a while loop to keep running until the user decides to exit the program (using the break statement).
- At the end of the program, the students’ names are printed with their corresponding accumulative average GPAs.

Grade Registry Program Tests

Test 1

Welcome to the Grade Registry Program

Would you like to add a new student? y(yes),n(no)

yEs

Enter the student's name:

Sarah Gray

Enter student GPA for each subject. Enter -1 to stop entering GPA.

2.3

4.0

-1

Would you like to add a new student? y(yes),n(no)

Ya

Incorrect Input, please enter y(yes)/n(no).

Would you like to add a new student? y(yes),n(no)

Yes

Enter the student's name:

John Doe

Enter student GPA for each subject. Enter -1 to stop entering GPA.

3.2

3.9

2.5

-1

Would you like to add a new student? y(yes),n(no)

YEs

Enter the student's name:

Steven Grant

Enter student GPA for each subject. Enter -1 to stop entering GPA.

-1

Would you like to add a new student? y(yes),n(no)

n

This is the list of students in the system, and their corresponding accumulative GPA

Sarah Gray 3.15

John Doe 3.2

Steven Grant 0

Marking Criteria

	Needs Improvement (0–50%)	Good (51–75%)	Excellent (76–100%)	Marks
Flow chart	- Largely incomplete - Poor structure	Good overall design, but not complete or there are steps missing	- Excellent design which can be followed to write a functional code - No missing steps or branches	/20
Working code	- The project doesn't run in all scenarios - Input requests work but don't match the scenario - Syntax of if/else statements has mistakes - Output works but doesn't match the scenario	- The project runs in all scenarios - Input requests work but don't match the scenario - Correct use of if/else statements - Output works but doesn't match the scenario	- The project runs in all scenarios - Input requests match the scenario exactly - Correct use of if/else statements - Output matches the scenario	/40
Style	- Indentation – not consistent - Readability – poor variable names - Documentation - No comments are included at the top. - No comments indicating major code sections or what they do	- Indentation – some parts are consistent and some are not - Readability – some variable names are not ideal - Documentation - Comments at the top are missing or incomplete. - Comments indicating major code sections and what they do are incomplete	- Indentation – consistent - Readability – good variable names - Documentation - Comments at the top are complete and include name, date, program description including details on inputs, processing and outputs (4–5 sentences minimum). - Comments indicate major code sections and what they do	/20

Testing	• Sample output doesn't match the provided test plan • Output is not formatted according to the specification (test plan)	• Parts of the sample output don't exactly match the provided test plan • Output formatted according to the specification (test plan)	• Sample output exactly matches the provided test plan • Output formatted according to the specification (test plan)	/20
Total				/100